

Dossier de projet

Projet Final - Développement Réseaux social

Auteur : CHIPAUX Laurent

Formation : Concepteur Développeur d'Applications

Technologies : Symfony

Repository GITHUB : [Reseau-social-CDA2025](#)

Remerciements :

Je tiens à remercier l'équipe pédagogique de Livecampus pour la qualité de l'enseignement et de l'accompagnement tout au long de cette formation.

Je remercie également l'agence La Couleur du Zèbre de m'avoir accueilli en tant qu'apprenti et de m'avoir fait confiance pour intégrer leurs projets. Je remercie particulièrement Emmanuel Winstein et l'ensemble de l'équipe pour leur accueil, leur patience et leurs précieux conseils.

1. Présentation de l'entreprise	3
2. Présentation du projet.	4
2.1. Contexte et problématique.	4
2.2. Objectifs du projet	4
2.3. Fonctionnalités principales	4
2.4. Contraintes techniques	5
3. Phase d'analyse	6
3.1. Introduction	6
3.2. Identification des utilisateurs cibles	6
3.3. Analyse documentaire.	7
3.4. User Stories	8
3.5. Diagramme de cas d'utilisation	9
3.6. Modélisation de la base de données	10
3.7. Choix techniques et justification.	12
3.8. Diagramme de classes	13
3.9. Diagramme d'activité	14
4. Gestion de projet.	16
4.1. Organisation et méthodologie	16
4.2. Découpage du projet	16
4.3. Planning prévisionnel (Gantt).	16
5. Maquettage	17
5.1. Wireframe	17
5.2. Identité visuelle	17
5.3. Composant	17

5.4. Maquettes haute fidélité	17
5.5. Responsive design	17
6. Développement	18
6.1. Back-end	18
6.1.1. Architecture du projet	18
6.1.2. Entités Doctrine	18
6.1.3. Authentification et sécurité	18
6.1.4. Contrôleurs principaux.	20
6.1.5. Interface d'administration (easyAdmin)	23
6.1.6. Requête personnalisées (Repository)	24
6.1.7. Gestion de l'upload des images	26
6.1.8. Test Unitaires.	27
6.2. Front-end	29
7. Déploiement	30
7.1. Conteneurisation avec Docker	30
7.2. Pipeline CI/CD avec GitLab	31
7.3. Processus de déploiement	32
7.4. Bilan du déploiement	32
8. Conclusion	33

1. Présentation de l'entreprise



La Couleur du Zèbre est une agence digitale basée à Strasbourg, spécialisée dans la conception de sites et d'applications web. Fondée en 2009, l'agence propose une expertise graphique, technique et stratégique. Ses services couvrent la stratégie digitale, la création de sites web, le graphisme, le conseil et la formation.

L'agence travaille principalement avec des entreprises et collectivités locales.

L'équipe est composée de 7 collaborateurs :

Emmanuel WINSTEIN : *Directeur / Chef de projet*

Sophie ENSMINGER : *Chef de projet / Chargée de communication*

Inès RENAUD : *Chef de projet*

Maryline WALDY : *Graphiste et webdesigner UX/UI*

Nina NACHEZ : *Graphiste et webdesigner UX/UI*

Nadège THIERRY : *Développeuse fullstack*

J'ai intégré l'équipe le **16/12/2024** dans le cadre de ma formation Concepteur Développeur d'Applications en tant qu'apprenti. Mon rôle au sein de l'entreprise a été d'intégrer différents projets en tant que développeur. J'ai participé à la conception de sites sur mesure Wordpress, la maintenance des sites existants et la communication avec les différents clients.

Voici quelques exemples de sites sur lesquels j'ai travaillé :

Alsace.com

Drulingen.fr

Cdq52.fr

Rhine-europe-terminals.com

Étant donné que j'ai principalement travaillé sur des sites utilisant le CMS Wordpress et au sein d'une équipe, je n'ai pu couvrir toutes les compétences attendues pour le titre professionnel **Concepteur Développeur d'Application**, c'est la raison pour laquelle j'ai choisis de faire ce dossier de soutenance sur un projet développé pendant ma formation.

2. Présentation du projet.

2.1. Contexte et problématique.

Dans le cadre de la formation Concepteur Développeur d'Applications, nous avons été amené à développer une application web répondant à un besoin concret : permettre aux habitants, associations et municipalités de découvrir et partager les événements locaux de leur territoire.

Le projet s'inscrit dans le contexte d'un grand journal régional souhaitant proposer une plateforme moderne de mise en relation autour des événements locaux, inspirée de services existants comme Infocale.

2.2. Objectifs du projet

L'application vise à offrir une expérience utilisateur simple et agréable, accessible via un site web responsive. Les objectifs principaux sont de permettre aux utilisateurs de consulter les événements de leurs villes et de leurs environs, de créer et gérer leurs propres événements, de personnaliser leur expérience en fonction des utilisateurs qu'ils suivent et de leurs villes de rattachement.

2.3. Fonctionnalités principales

Gestion des événements :

Chaque événement est composé d'un titre, d'une description, d'un lieu, d'une date et peut contenir plusieurs photos. Il est rattaché à une ville et peut appartenir à un ou plusieurs types (concert, sport, marché, etc). Les événements sont affichés du plus proche au plus lointain dans le temps. Les événements passés ne sont plus visibles publiquement mais restent accessibles à leur créateur.

Gestion des utilisateurs :

Un utilisateur connecté peut sélectionner plusieurs villes de rattachement afin de personnaliser son fil d'actualité. Il peut également suivre des créateurs d'événements favoris pour affiner davantage les contenus proposés sur sa page d'accueil.

Fonctionnalités premium :

Une option payante permet aux utilisateurs de mettre en avant leurs événements pour gagner en visibilité.

Interface d'administration :

Un tableau de bord administrateur offre des statistiques détaillées : nombre d'utilisateurs et d'événements au global, par département et par ville. L'administrateur peut également consulter les informations des utilisateurs, incluant leurs données personnelles, le nombre d'événements créés et les dépenses effectuées pour la mise en avant.

2.4. Contraintes techniques

Ce projet a été réalisé en respectant les exigences du titre professionnel **Concepteur Développeur d'Applications**, couvrant l'ensemble du cycle de développement.

Analyse et conception : élaboration du MCD/MLD, diagrammes UML.

Gestion de projet : méthodologie agile.

Maquettage : Wireframes, définition de l'identité visuelle, conception des composants, maquettes haute fidélité et gestion du responsive design.

Développement : développement frontend, backend.

Déploiement : mise en place de tests automatisés, Dockerisation de l'application et configuration d'une pipeline CI/CD pour un déploiement continu.

3. Phase d'analyse

3.1. Introduction

J'ai développé ce projet en collaboration avec deux autres apprenants. Ce qui nous a permis de nous confronter aux réalités du travail collaboratif : répartition des tâches, communication, gestion des versions avec Git et coordination des développements.

Chaque membre de l'équipe a contribué aux différentes phases du projet, de l'analyse des besoins jusqu'au déploiement. Cette expérience collective a enrichi notre apprentissage en nous confrontant aux problématiques de synchronisation du code.

3.2. Identification des utilisateurs cibles

Afin de mieux comprendre les besoins et les attentes des futurs utilisateurs de la plateforme, nous avons élaboré trois personas représentatifs des différents profils identifiés.

Léa CHEMIN incarne l'utilisateur classique, à la recherche d'événements locaux pour ses loisirs. Elle représente une cible jeune, connectée, qui souhaite découvrir facilement ce qui se passe autour d'elle.

Sarah GARCIA représente le profil créateur d'événements : professionnelle ou membre d'association, elle a besoin d'un outil simple pour publier ses événements et toucher un public local.

Victor MARIN : correspond au profil administrateur de la plateforme. En tant que développeur, il attend un outil stable, bien organisé et facile à modérer.

Léa Chemin



AGE 19
DIPLOME Etudiante en BTS Audiovisuel
STATUT En couple
POSTE Etudiante
VILLE Toulouse
NIVEAU TECH. Très bon

“ Je veux avoir facilement et rapidement accès à ce que je cherche, sans devoir me créer de compte pour faire le moindre truc. ”

Personnalité

Extravertie
Toujours partante Fêtarde

Bio & Objectifs

Etudiante en 2e année de BTS audiovisuel, elle s'intéresse à divers événements de sa commune : pièces de théâtre, projection cinéma mais aussi concerts et stand-up. Elle souhaite pouvoir avoir accès à une liste complète des événements autour de chez elle afin de pouvoir y accéder facilement. tout en respectant son budget serré.

Besoins

- Savoir ce qu'il se passe ce week-end
- Trouver des concerts de ses artistes favoris
- Filtrer par catégorie
- Accéder rapidement à l'information recherchée

Frustrations

- Peur de rater un événement en n'ayant pas entendu parler
- Trop d'applications différentes pour faire ses recherches
- Obligation de créer un compte pour accéder à des fonctionnalités simples
- Les informations des événements passées ou pas à jour

Sarah Garcia



AGE 29
DIPLOME Master en communication
STATUT Célibataire
POSTE Chargée de communication dans une PME
VILLE Toulouse
NIVEAU TECH. Bon niveau

“ J'ai besoin d'un outil simple, clair et professionnel pour me permettre de facilement créer et suivre mes événements. ”

Personnalité

Extravertie Sportive
Joyeuse

Bio & Objectifs

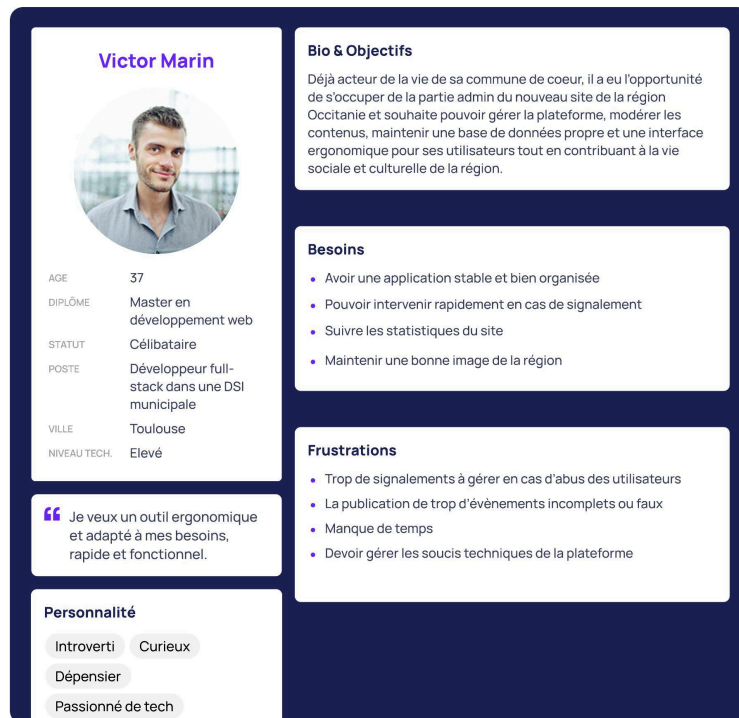
Chargée de communication dans une PME Toulousaine, elle souhaite à terme ouvrir sa propre entreprise de conseil en communication aux entreprises. A côté de cela, elle organise régulièrement des événements tels que des rencontres autour de la communication ainsi que des ateliers de poterie, sa passion. Elle aime également assister à divers événements.

Besoins

- Avoir un endroit où créer et promouvoir ses événements
- Toucher un public plus large
- Partager ses passions
- Développer son réseau
- Avoir un flux personnalisé

Frustrations

- Le manque de visibilité des événements locaux sur les réseaux classiques
- La difficulté à toucher un public plus large
- Le manque d'ergonomie et la complexité de certains sites
- Le manque de suivi sur la participation à ses événements



Ces trois personas nous ont guidés tout au long de la conception pour garantir une expérience adaptée à chaque type d'utilisateur.

3.3. Analyse documentaire.

Avant de débuter la conception de l'application, nous avons réalisé une étude comparative de plateformes existantes proposant des fonctionnalités similaires. Cette analyse nous a permis d'identifier les bonnes pratiques en matière d'expérience utilisateur, de comprendre les fonctionnalités essentielles attendues par les utilisateurs, de repérer les points faibles à éviter et de définir des axes de différenciation pour notre projet.

Nous avons principalement étudié les plateformes suivantes :

<https://shotgun.live/fr> : Barre de recherche avec filtre / visuel des articles / thèmes sombre / Accueil en fonction de la localisation

<https://www.bigcitynantes.fr/> : Articles sponsorisé

<https://www.sortiraparis.com/> : Trop d'informations sur les pages du au pub

<https://pokaa.fr/> : Logo / simplicité des pages

3.4. User Stories

À partir des personas définis et de l'analyse des besoins, nous avons rédigé les **user stories** du projet. Ces récits utilisateurs permettent d'exprimer les fonctionnalités attendues du point de vue des différents acteurs de la plateforme.

Chaque user story suit le format standard : "En tant que [rôle], je veux [action] afin de [bénéfice]". Cette approche nous a permis de garder une vision centrée sur l'utilisateur tout au long du développement et de prioriser les fonctionnalités à implémenter.

Nous avons identifié trois types d'acteurs :

- l'utilisateur non connecté (visiteur)
- l'utilisateur connecté (créateur d'événements)
- l'administrateur

En tant qu'utilisateur non connecté

Je veux rechercher des événements par ville et par catégorie
Afin de trouver rapidement des activités qui m'intéressent.

En tant qu'utilisateur connecté

Je veux créer un événement avec titre, description, date et lieu
Afin de promouvoir mes activités auprès du public local.

En tant qu'utilisateur connecté

Je veux mettre en avant mon événement (fonctionnalité payante)
Afin d'augmenter sa visibilité et attirer plus de participants.

En tant qu'utilisateur connecté

Je veux suivre mes villes préférées
Afin de voir leurs nouveaux événements en priorité sur ma page d'accueil

En tant qu'utilisateur connecté

Je veux suivre mes créateurs d'événements préférés
Afin de voir leurs nouveaux événements en priorité sur ma page d'accueil

En tant qu'administrateur

Je veux valider les demandes de promotion des événements
Afin de contrôler les événements mis en avant.

En tant qu'administrateur

Je veux pouvoir gérer les données du site
Afin de suivre l'activité de la plateforme

3.5. Diagramme de cas d'utilisation

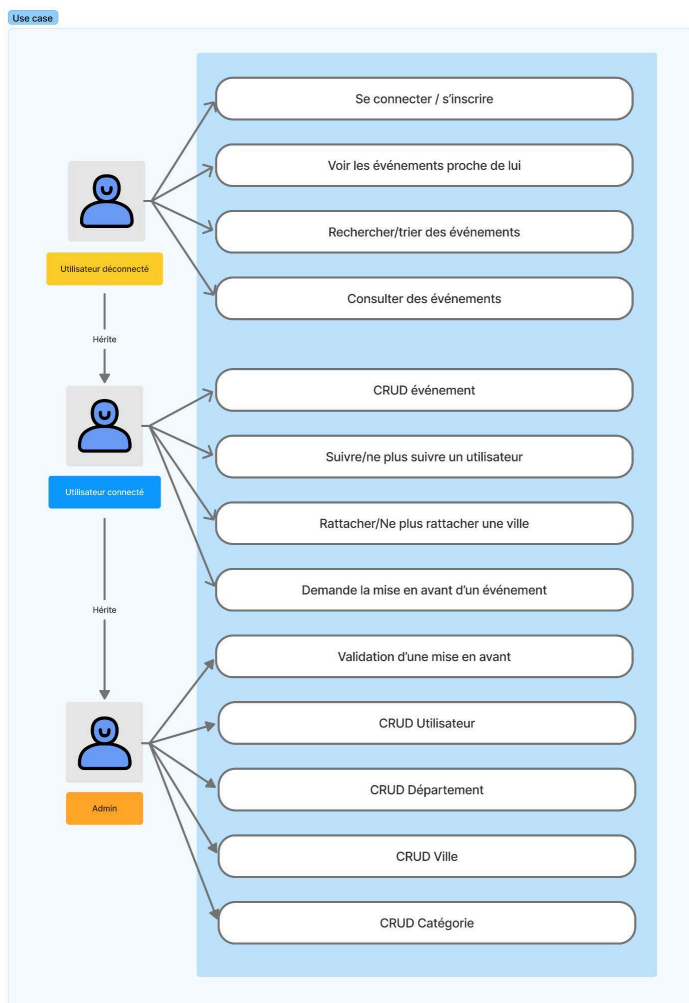
Le diagramme de cas d'utilisation permet de représenter les interactions entre les différents acteurs et le système. Il offre une vision globale des fonctionnalités accessibles selon le niveau de droits de chaque utilisateur

Nous avons identifié trois acteurs principaux avec une relation d'héritage entre chaque :

L'utilisateur déconnecté dispose des fonctionnalités de base : il peut consulter les événements, effectuer des recherches par ville ou catégorie, et accéder à la page de connexion ou d'inscription.

L'utilisateur connecté hérite des droits de l'utilisateur déconnecté et peut en plus créer, modifier ou supprimer ses événements, suivre d'autres utilisateurs, se rattacher à des villes et demander la mise en avant de ses événements.

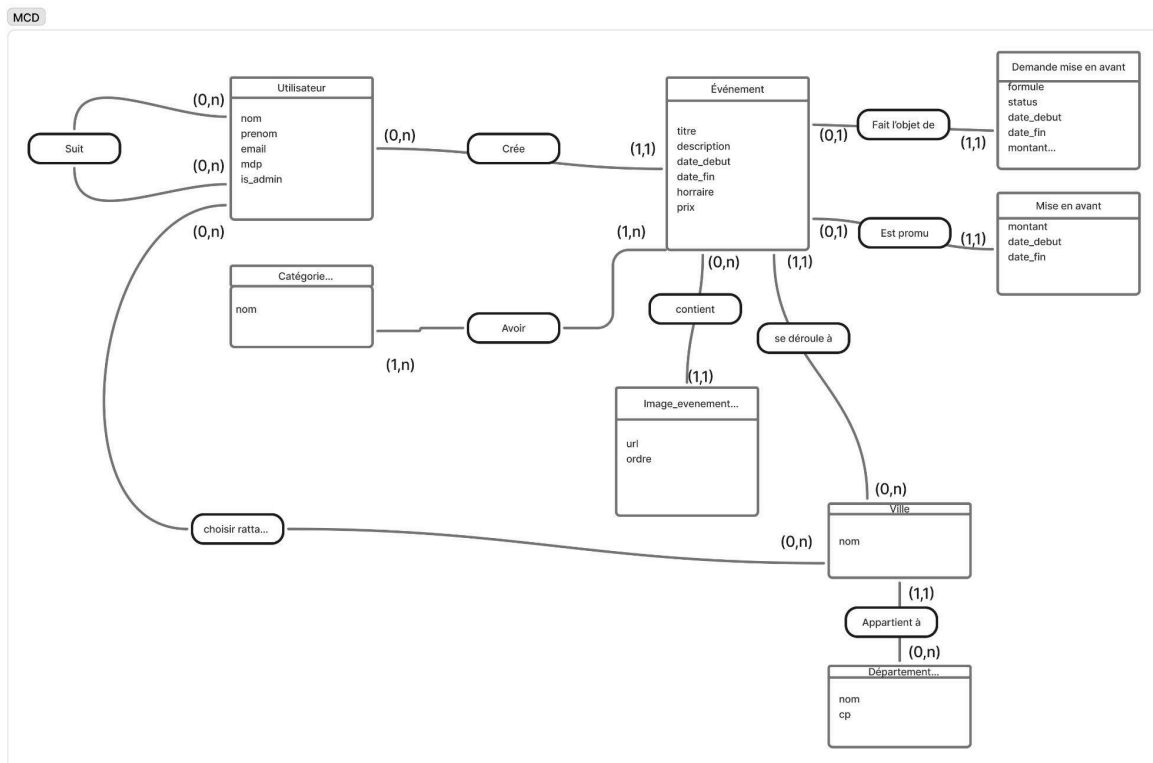
L'administrateur hérite des droits de l'utilisateur connecté et dispose de fonctionnalités supplémentaires de gestion : validation des demandes de mise en avant, et administration des utilisateurs, départements, villes et catégories (CRUD)



3.6. Modélisation de la base de données

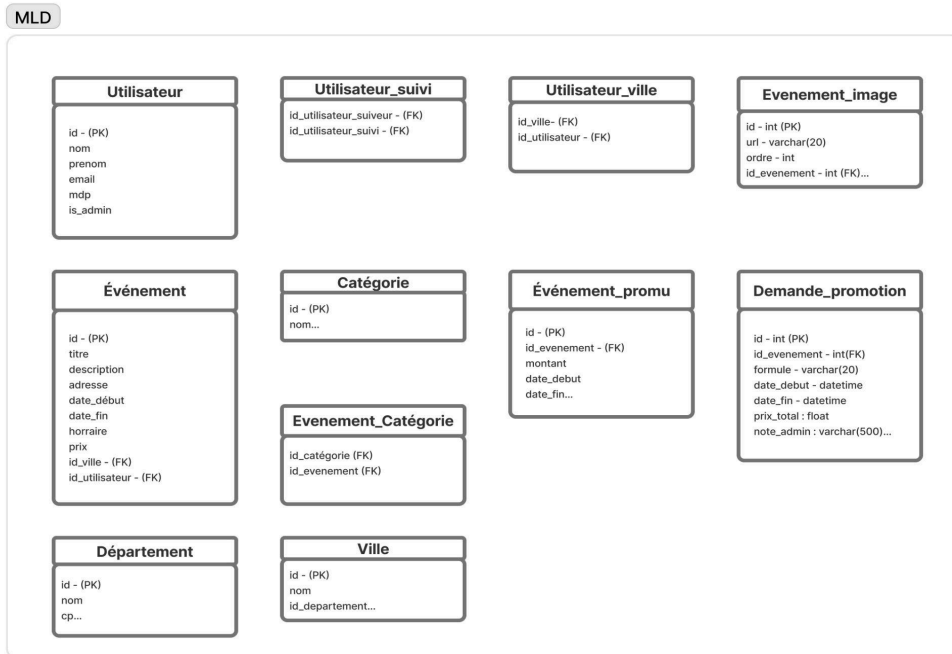
3.6.1. Modèle Conceptuel de Données (MCD)

Notre modèle s'articule autour de plusieurs entités principales : Utilisateur, Événement, Ville, Département et Catégorie. Les relations entre ces entités reflètent les fonctionnalités de l'application : un utilisateur peut créer plusieurs événements, suivre d'autres utilisateurs, se rattacher à des villes, et demander la mise en avant de ses événements.



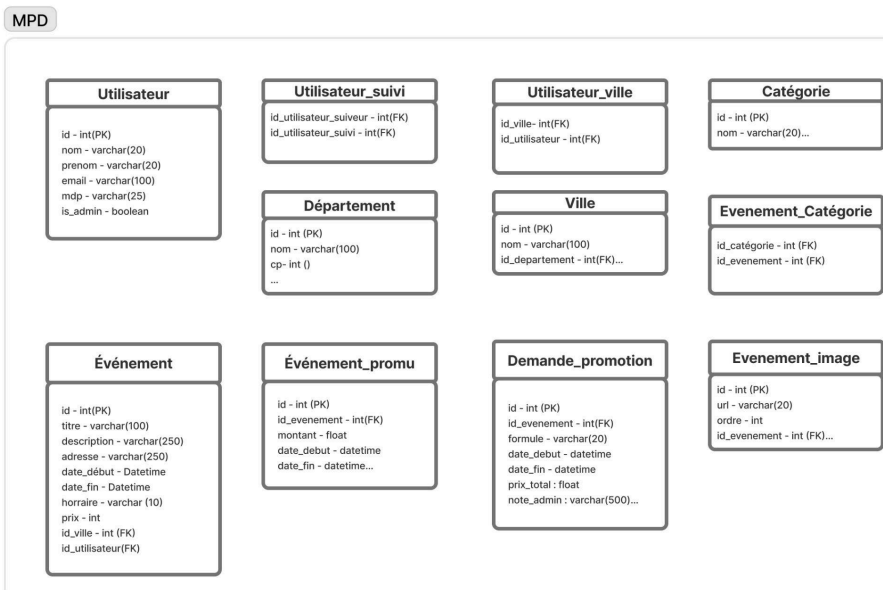
3.6.2. Modèle Logique de Données (MLD)

Le MLD est la traduction du MCD en un schéma relationnel. Les entités deviennent des tables, les associations sont transformées en clés étrangères ou en tables de liaison pour les relations Many-to-Many



3.6.3. Modèle Physique de Données (MPD)

Le MDP précise les types de données de chaque attribut et prépare l'implantation concrète en base de données.



3.7. Choix techniques et justification.

Le choix des technologies a été guidé par plusieurs critères : la maîtrise de l'équipe, la productivité, la maintenabilité du code et les exigences du titre professionnel.

Symfony :

Nous avons choisi le framework Symfony pour plusieurs raisons. Tout d'abord, les trois membres de l'équipe étaient à l'aise avec cet environnement, ce qui nous a permis d'être plus facilement opérationnels. Symfony offre également un système de sécurité robuste et intégré (authentification, gestion des rôles, protection CSRF), nous évitant de développer ces fonctionnalités de zéro. Nous avons utilisé le bundle EasyAdmin pour générer rapidement l'interface d'administration, ce qui nous a fait gagner un temps considérable sur le back-office. Enfin, l'intégration native de PHPUnit facilite la mise en place de test unitaires et fonctionnels, une exigence du titre professionnel.

MySQL :

MySQL a été retenu pour sa compatibilité avec Symfony via Doctrine ORM. L'utilisation de Doctrine nous permet de manipuler les données sous forme d'objets PHP et de générer automatiquement les migrations.

Figma :

Figma s'est imposé comme l'outil de conception idéal pour un travail collaboratif, l'ensemble de l'équipe pouvait travailler simultanément sur les maquettes en temps réel. Nous l'avons également utilisé pour créer nos schémas (Personas, diagrammes) afin de centraliser tous nos livrables visuels au même endroit.

Gitlab :

Gitlab a été choisi pour centraliser le code source, gérer les branches et le versioning mais aussi pour sa fonctionnalité de pipeline CI/CD intégré. Cela nous a permis d'automatiser les tests et le déploiement.

Docker :

Docker nous a permis d'uniformiser les environnements de développement entre les trois membres de l'équipe, évitant ainsi les problèmes de configuration. Il facilite également le déploiement en production en garantissant un environnement identique.

3.8. Diagramme de classes

Le diagramme de classes représente la structure objet de l'application. Dans le contexte de Symfony et Doctrine ORM, il reflète directement les entités PHP qui seront persistées en base de données.

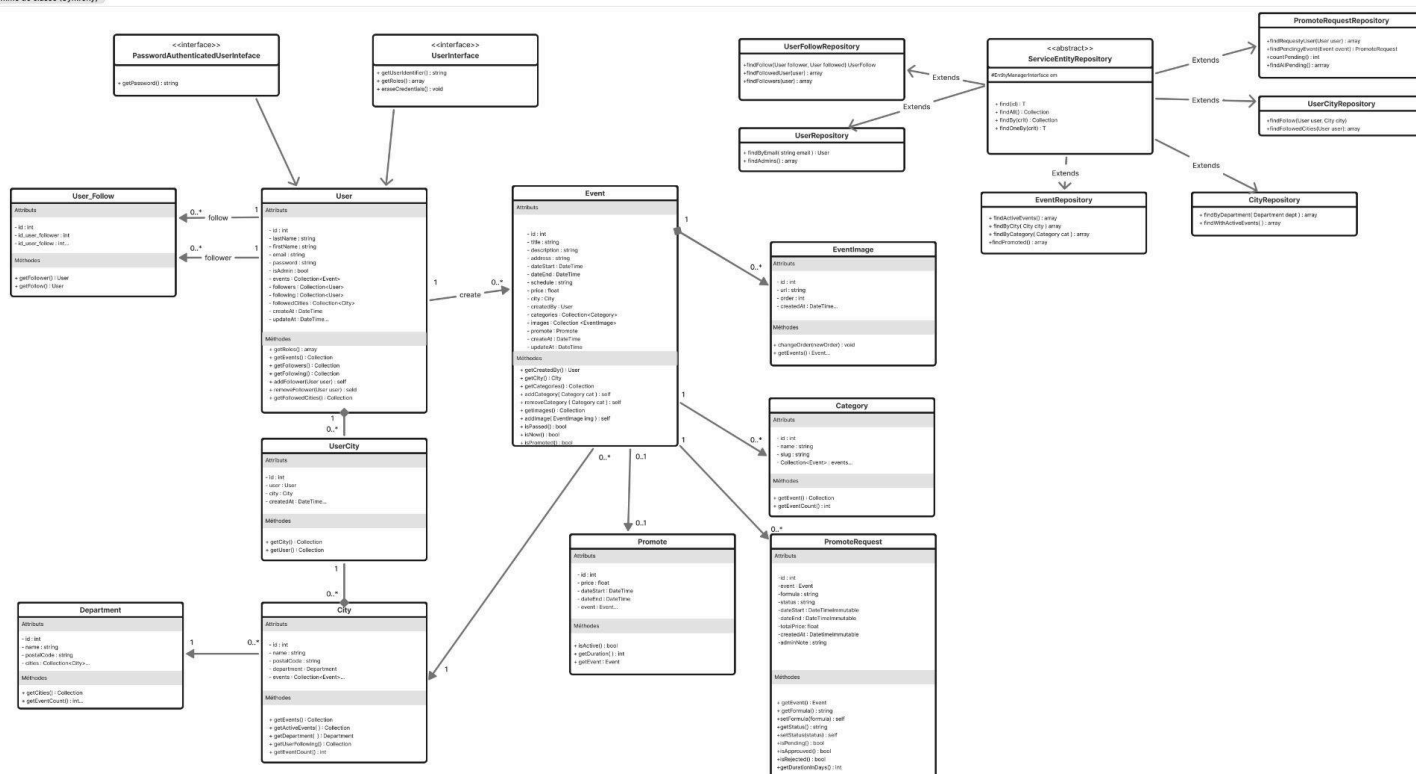
Les entités principales :

- **User** : représente un utilisateur de la plateforme. Elle implémente les interfaces "UserInterface" et "PasswordAuthenticatedUserInterface" fournies par Symfony pour gérer l'authentification et la sécurité.
- **Event** : représente un événement avec ses attributs (titre, description, adresse, dates, prix) et ses relations avec l'utilisateur créateur, la ville, les catégories et les images
- **User_Follow** : table de liaison permettant de gérer le système de suivi entre utilisateurs (follower / following)
- **UserCity** : table de liaison entre les utilisateurs et leurs villes de rattachement.
- **City** et **Department** : gèrent la localisation géographique des événements selon une hiérarchie département/ville
- **Category** : permet de classer les événements par type.
- **EventImage** : stocke les images associées à un événement.
- **Promote** et **PromoteRequest** : gèrent le système de mise en avant payante des événements et les demande de mise en avant.

Les Repositories :

Chaque entité dispose d'un repository qui hérite de "ServiceEntityRepository". Ces classes contiennent les méthodes de requêtes personnalisées pour accéder aux données.

Diagramme de classe (Symfony)



3.9. Diagramme d'activité

Le diagramme d'activité permet de représenter le déroulement d'un processus métier étape par étape. Il met en évidence les conditions, les actions et les différents chemins possibles selon les décisions prises.

Nous avons choisi de modéliser le processus de création d'un événement, qui est la fonctionnalité principale de l'application.

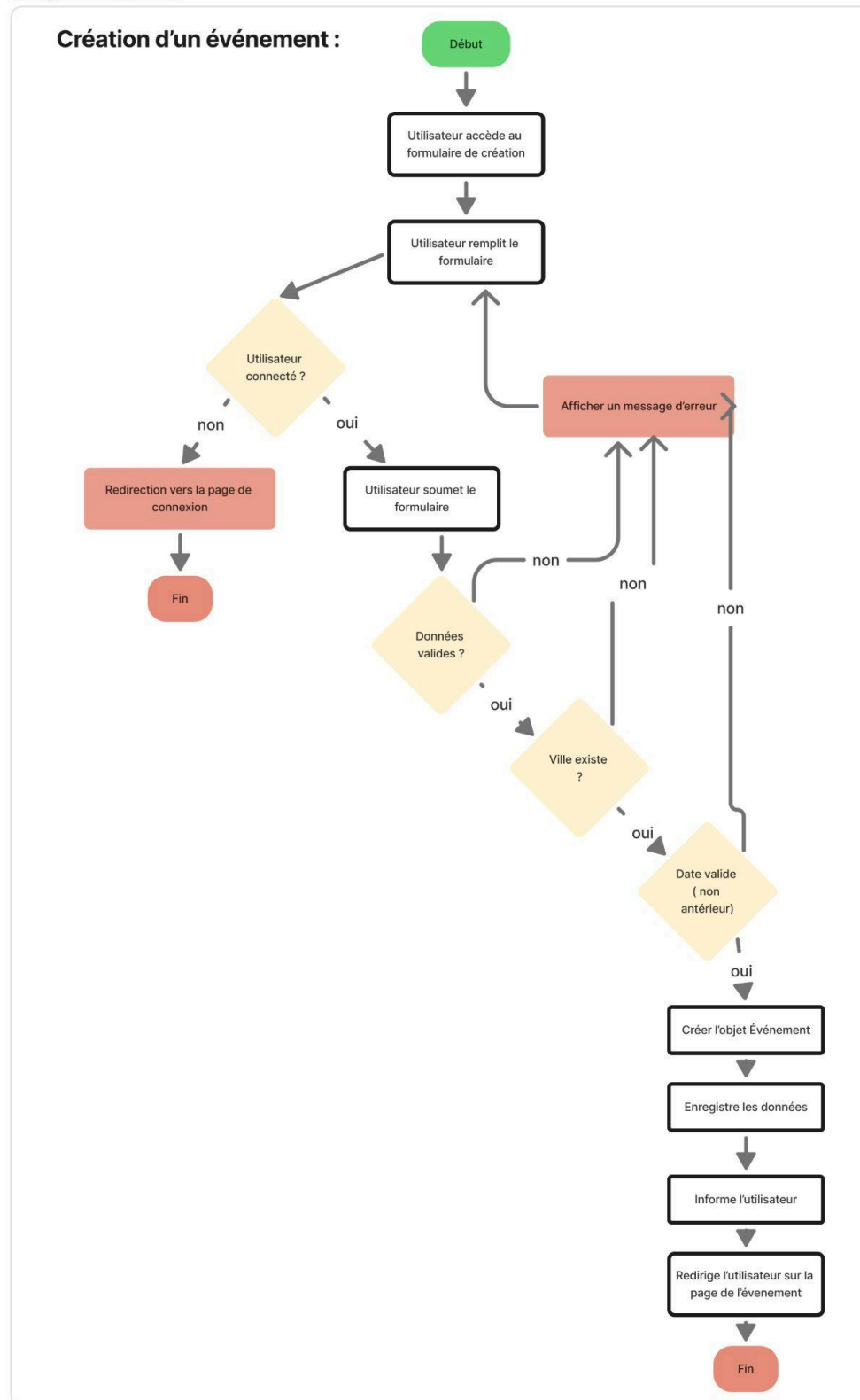
Création d'un événement :

Ce diagramme illustre le parcours complet d'un utilisateur souhaitant créer un événement. Le processus commence par l'accès au formulaire de création et inclut plusieurs points de validation :

- Vérification de l'authentification de l'utilisateur
- Validation des données saisies dans le formulaire
- Vérification de l'existence de la ville sélectionnée
- Contrôle de la cohérence de la date (non antérieure à la date actuelle)

En cas d'échec à l'une des étapes, un message d'erreur est affiché et l'utilisateur est invité à corriger sa saisie. Si toutes les validations sont réussies, l'événement est créé, enregistré en base de données, et l'utilisateur est redirigé vers la page de son événement.

Diagramme d'activité



4. Gestion de projet.

4.1. Organisation et méthodologie

Le projet s'est déroulé sur période d'environ quatre semaines, du 24 novembre au 18 décembre 2025. Pour organiser notre travail en équipe de trois développeurs, nous avons adopté une approche agile avec un suivi des tâches via l'outil ClickUp, qui nous permettait de visualiser l'avancement grâce à un diagramme de Gantt.

4.2. Découpage du projet

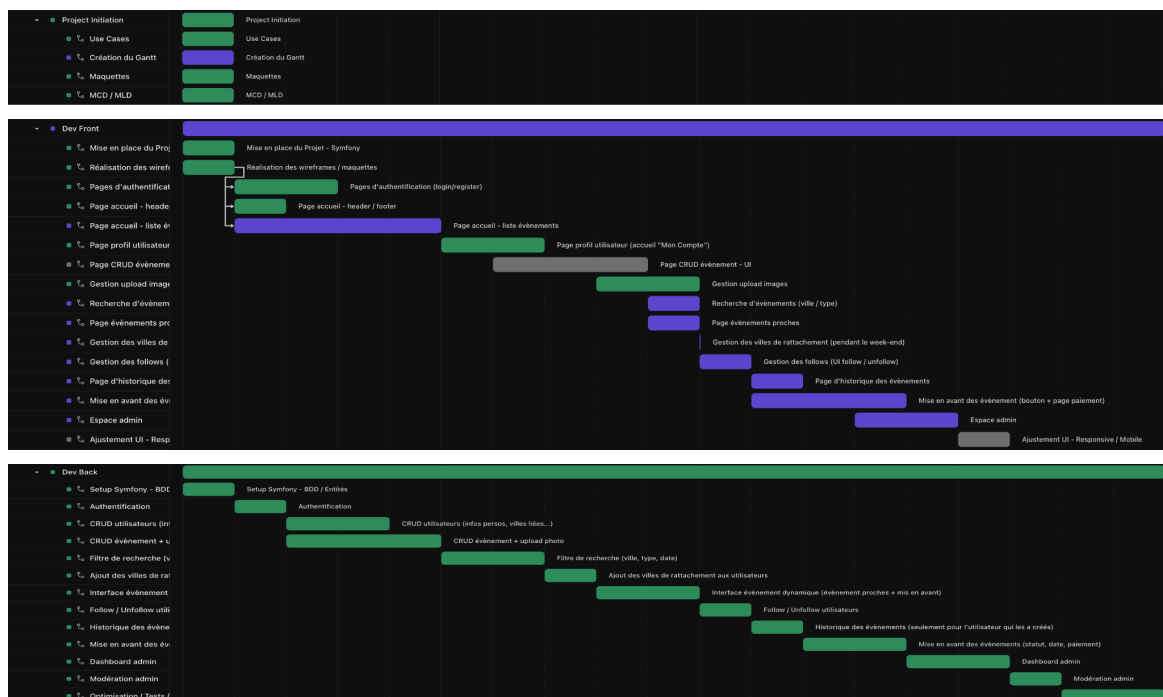
Nous avons structuré le projet en trois grandes phases :

Phase d'initiation : Cette première phase a permis de poser les bases du projet avec la rédaction des cas d'utilisation, la création du planning Gantt, la réalisation des maquettes et la modélisation de la base de données.

Développement back-end : Cette phase comprenait la mise en place du projet Symfony et des entités, le système d'authentification, les CRUD utilisateurs et événements, les filtres de recherche, la gestion des villes de rattachement, le système de follow/unfollow, la mise en avant des événements, ainsi que le dashboard et la modération admin.

Développement Front-end : En parallèle du back-end, nous avons travaillé sur l'intégration des Wireframes et des maquettes, les pages d'authentification, la page d'accueil, avec la liste des événements, le profil utilisateur, et les ajustements pour le responsive.

4.3. Planning prévisionnel (Gantt).



5. Maquettage

L'ensemble du maquettage a été réalisé avec Figma, un outil collaboratif permettant de travailler en équipe et de maintenir une cohérence visuelle grâce aux composants réutilisables.

5.1. Wireframe

Les wireframes constituent la première étape de conception. Ces maquettes basse fidélité, en niveaux de gris, permettent de définir la structure des pages et la hiérarchie de l'information sans se soucier du design visuel.

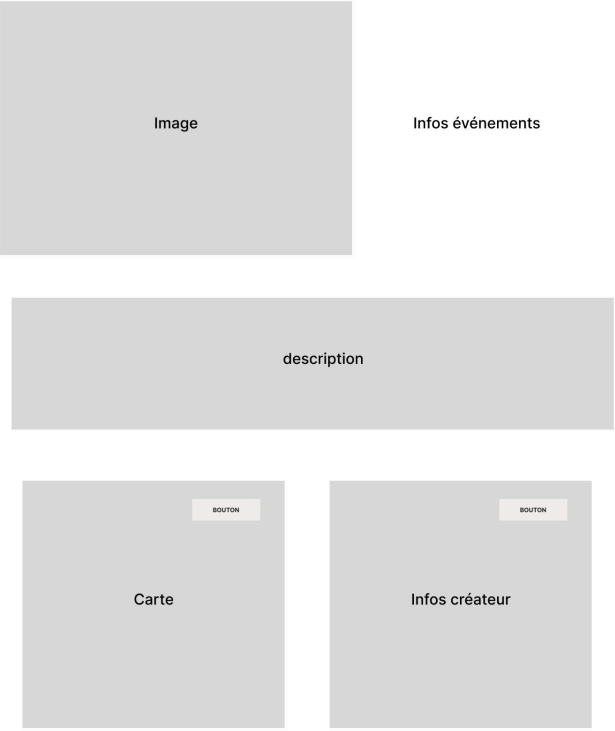
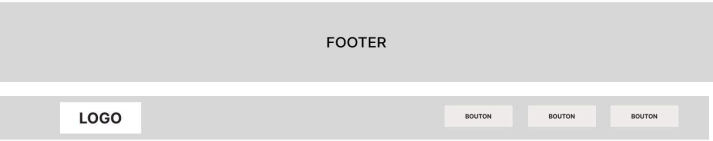
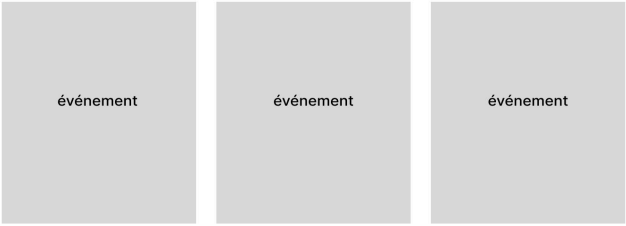
Nous avons réalisé des wireframes pour les pages principales de l'application :

Page d'accueil : Un hero avec un formulaire de recherche rapide (texte, catégorie, département), suivi d'une section d'inscription et d'une grille d'événement.

Page liste des événements : Un système de filtres latéral (recherche textuelle, catégories, dates, département, ville, événements gratuits) accompagné d'une grille de cards événements sur 3 colonnes.

Page détail événement : Une mise en page en deux colonnes avec l'image à gauche et les informations principales à droite, suivi d'une section description et de blocs pour la localisation avec une carte et les informations sur l'organisateur.

Tableau de bord utilisateur : Un message de bienvenue personnalisé, quatre cards de statistiques (événements créés, abonnés, villes suivies, abonnements), et deux sections présentant les événements des villes suivies et l'activité des abonnements.



filtre

événement

événement

événement

événement

événement

événement

Lorem ipsu
 Lorem ipsum dolor sit amet, consectetur
 adipiscing elit, sed do eiusmod tempor

1

5

6

3

événement
ville suivi

BOUTON

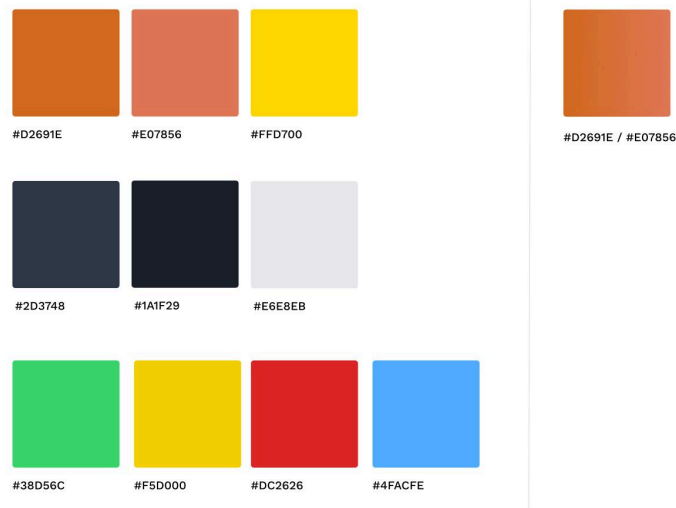
événement
créateur suivi

BOUTON

5.2. Identité visuelle

Avant de créer les maquettes haute fidélité, nous avons défini un design system garantissant la cohérence visuelle de l'application.

5.2.1. Palette de couleurs



5.2.2. Typographies

Deux familles typographiques ont été sélectionnées :

- **Vast Shadow** : Police décorative utilisée exclusivement pour le logo “La Vie en Rose”.
- **Inter** : Police sans-serif moderne et très lisible, utilisé pour l'ensemble du contenu

Vast Shadow / regular / logo

La Vie en Rose

Inter / bold / h1

52px

Titre h1

Inter / semi-bold / h2

28px

Titre H2

Inter / semi-bold / h3

18px

Titre H3

Inter / Regular / Texte courant

15px

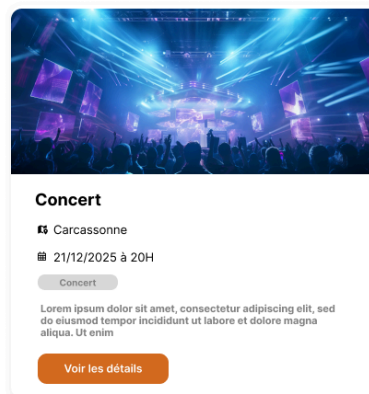
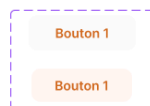
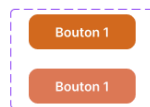
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis venenatis, erat ac ullamcorper pharetra, arcu metus commodo nunc, ut aliquam lorem lectus at lacus.

5.2.3. Iconographie

Nous avons utilisés Font Awesome pour définir des icônes cohérents pour différentes fonctionnalités



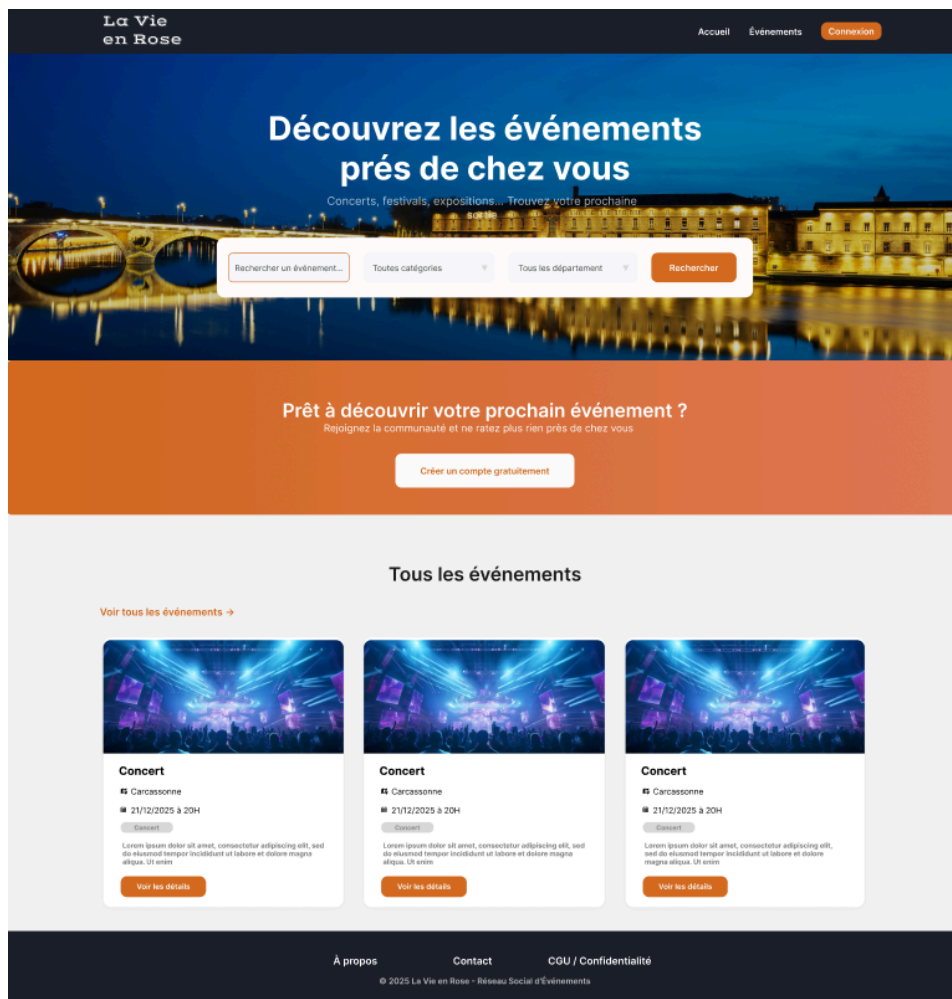
5.2.4. Composants



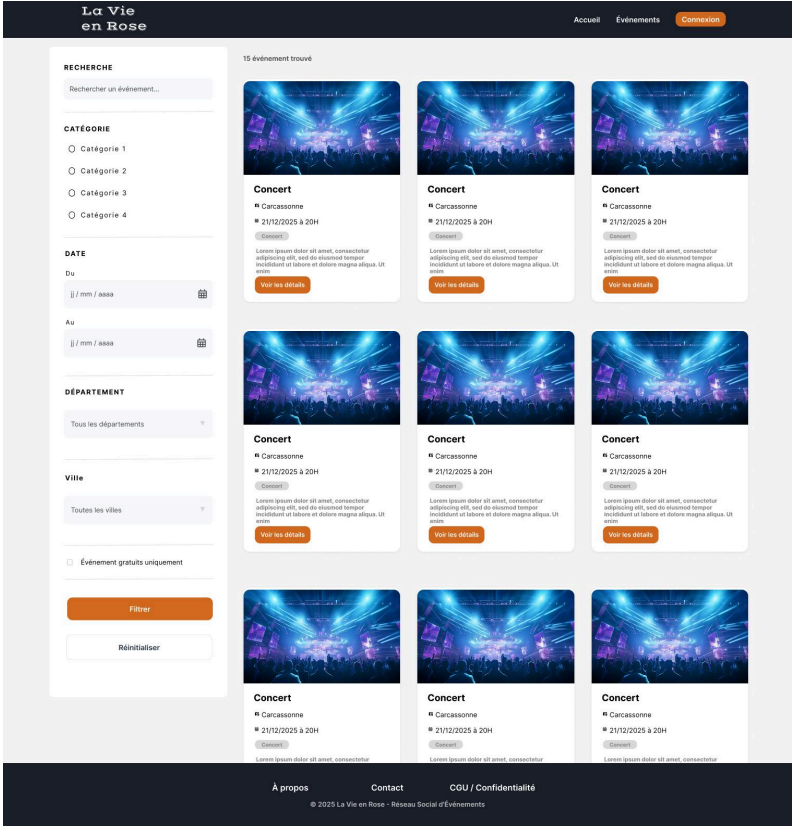
5.3. Maquettes haute fidélité

Les maquettes fidélité représentent le rendu final de l'application en appliquant le design system aux wireframes.

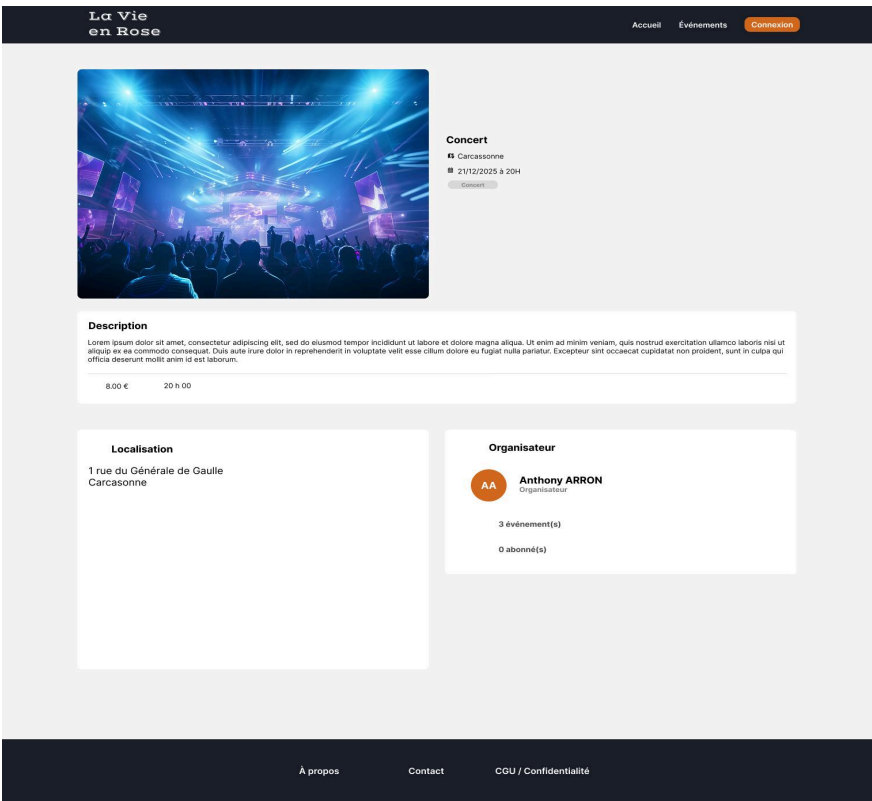
5.3.1. Page d'accueil



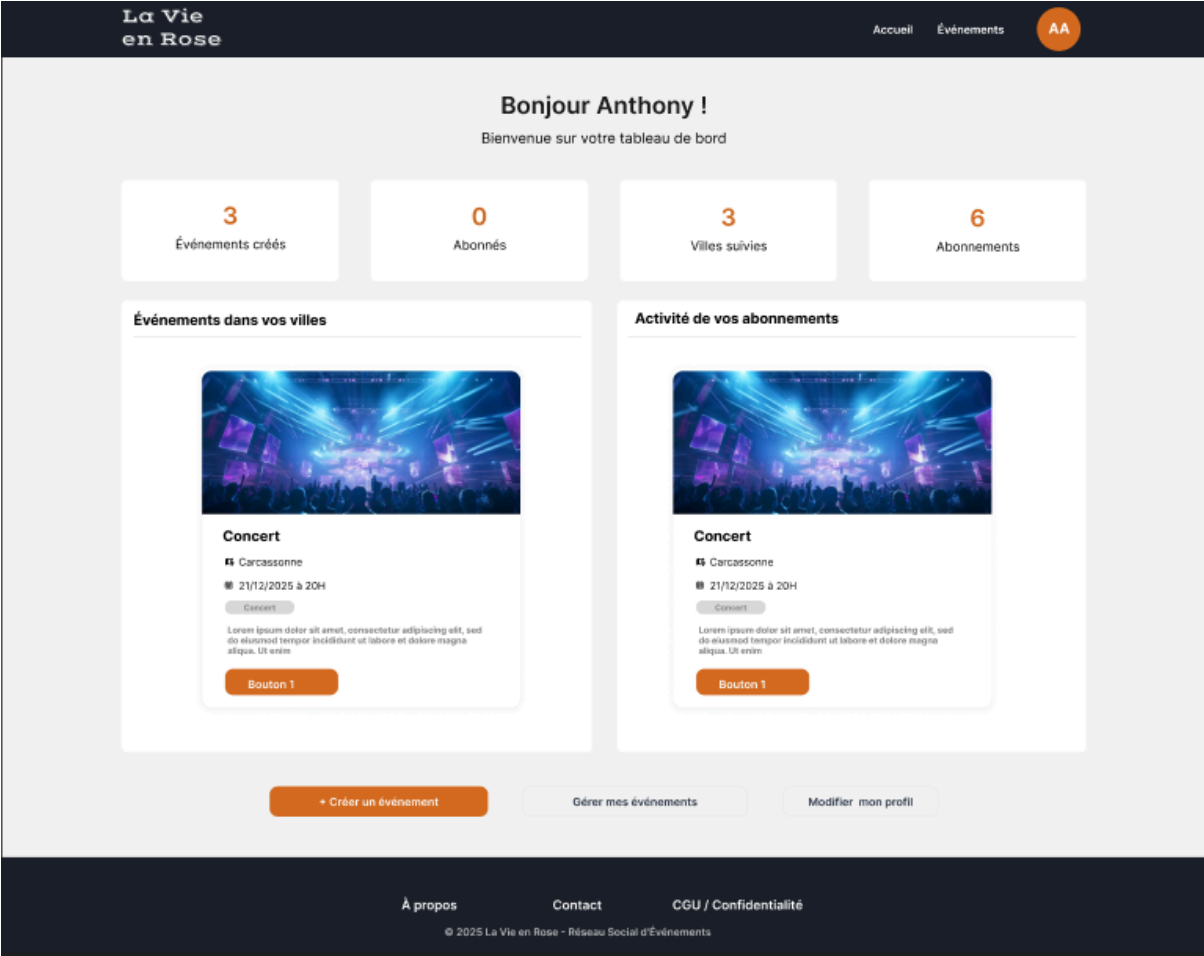
5.3.2. Page liste des événements.



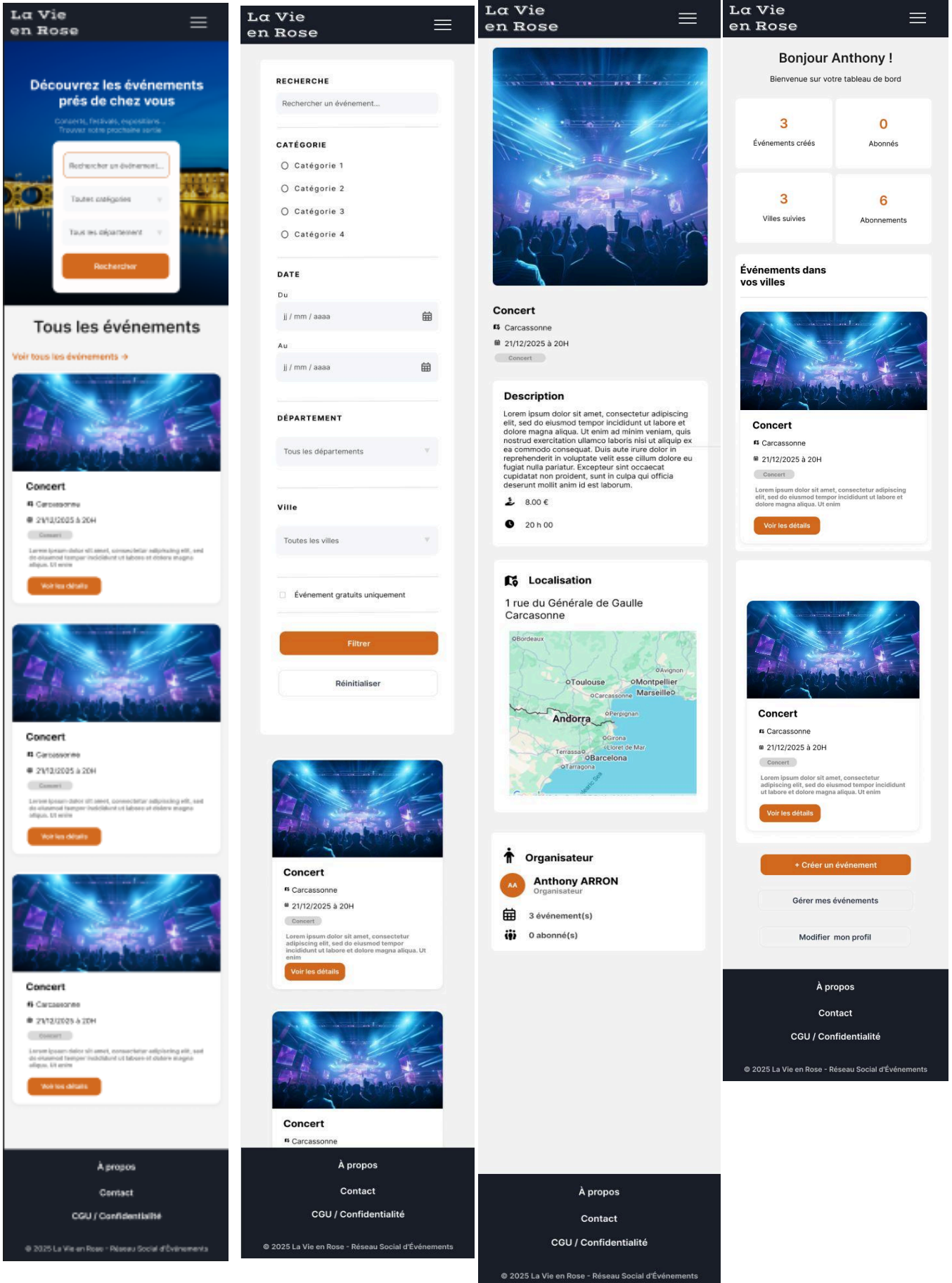
5.3.3. Page détail événement



5.3.4. Tableau de bord utilisateur



5.4. Responsive design



6. Développement

6.1. Back-end

6.1.1. Architecture du projet

Le projet suit l'architecture standard de Symfony avec une séparation claire des responsabilités: les controllers gèrent les requêtes HTTP, les entités représentent le modèle de données, les repositories encapsulent les requêtes à la base de données, les formulaires gèrent la validation et le traitement des données utilisateur, et les services contiennent la logique métier.

6.1.2. Entités Doctrine

Le modèle de données comprend 10 entités :

- **User** : gestion des utilisateurs avec authentification Symfony Security
- **Event** : les événements avec leurs attributs (titre, description, dates, prix, adresse)
- **City** et **Department** : hiérarchie géographique pour la localisation
- **Category** : classification des événements
- **EventImage** : gestion des images multiples par événement
- **UserFollow** : système de suivi entre utilisateurs
- **UserCity** : villes de rattachement des utilisateurs
- **Promote** et **PromoteRequest** : système de mise en avant payante

6.1.3. Authentification et sécurité

L'authentification repose sur le composant Symfony Security avec un authenticator personnalisé "AppAuthenticator". La configuration définit plusieurs niveaux d'accès via les paramètres "access_control". Les routes "/admin" sont réservées aux utilisateurs ayant le rôle "ROLE_ADMIN", les routes "/profile" et "/dashboard" nécessitent le rôle "ROLE_USER", et les pages de connexion et d'inscription sont rendues public.

La protection CSRF est activée sur tous les formulaires, et le hachage des mots de passe utilise l'algorithme par défaut de Symfony (bcrypt/argon2).

Fichier de configuration security.yaml :

```
security:
  ⚡ # Configuration du hashage des mots de passe
  # 'auto' laisse Symfony choisir le meilleur algorithme (bcrypt ou argon2)
  password_hashers:
    Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'

  # Provider : indique où trouver les utilisateurs
  providers:
    app_user_provider:
      entity:
        class: App\Entity\User
        property: email

  firewalls:
    main:
      lazy: true
      provider: app_user_provider
      # Authenticator personnalisé pour gérer la connexion
      custom_authenticator: App\Security\AppAuthenticator
      logout:
        path: app_logout
        target: app_login
      # Fonctionnalité "Se souvenir de moi"
      remember_me:
        secret: '%kernel.secret%'
        lifetime: 604800 # 7 jours en secondes

  # Contrôle d'accès : définit les permissions par route
  access_control:
    - { path: ^/admin, roles: ROLE_ADMIN } # Admin uniquement
    - { path: ^/dashboard, roles: ROLE_USER } # Utilisateurs connectés
    - { path: ^/profile, roles: ROLE_USER } # Utilisateurs connectés
    - { path: ^/login, roles: PUBLIC_ACCESS } # Accessible à tous
    - { path: ^/register, roles: PUBLIC_ACCESS } # Accessible à tous
```

6.1.4. Contrôleurs principaux.

EventController

Ce contrôleur gère l'ensemble des opérations liées aux événements : affichage, recherche, création, modification, suppression.

La méthode "index" implémente le système de filtres multicritères permettant de rechercher des événements par ville, département, catégorie, date, et prix.

```
#[Route('/', name: 'app_event_index')]
public function index(
    Request $request,
    EventRepository $eventRepository,
    CategoryRepository $categoryRepository,
    DepartmentRepository $departmentRepository,
    CityRepository $cityRepository
): Response {
    // Récupère les filtres depuis la requête
    $filters = [
        'city' => $request->query->get( key: 'city'),
        'department' => $request->query->get( key: 'department'),
        'category' => $request->query->get( key: 'category'),
        'dateFrom' => $request->query->get( key: 'date_from'),
        'dateTo' => $request->query->get( key: 'date_to'),
        'freeOnly' => $request->query->getBoolean( key: 'free_only'),
        'search' => $request->query->get( key: 'search'),
    ];

    $events = $eventRepository->findWithFilters(
        cityId: $filters['city'] ? (int) $filters['city'] : null,
        departmentId: $filters['department'] ? (int) $filters['department'] : null,
        categoryId: $filters['category'] ? (int) $filters['category'] : null,
        $filters['dateFrom'],
        $filters['dateTo'],
        $filters['freeOnly'],
        $filters['search']
    );

    return $this->render( view: 'event/index.html.twig', [
        'events' => $events,
        'categories' => $categoryRepository->findBy([], ['name' => 'ASC']),
        'departments' => $departmentRepository->findBy([], ['name' => 'ASC']),
        'cities' => $cityRepository->findBy([], ['name' => 'ASC']),
        'currentFilters' => $filters,
    ]);
}
```

ProfileController

Ce contrôleur centralise toutes les fonctionnalités du compte utilisateur : modification du profil, gestion des événements créés, upload d'images et demandes de mise en avant.

Il gère notamment l'upload des images et la suppression des images lorsqu'un événement est supprimé :

```
#[Route('/events/{id}/delete', name: 'app_profile_event_delete', requirements: ['id' => '\d+'], methods: ['POST'])]
public function deleteEvent(
    Event $event,
    Request $request,
    EntityManagerInterface $em
): Response {
    if ($event->getCreatedBy() !== $this->getUser()) {
        $this->addFlash( type: 'error', message: 'Vous ne pouvez pas supprimer cet événement. ');
        return $this->redirectToRoute( route: 'app_profile_events' );
    }

    if ($this->isCsrfTokenValid( id: 'delete' . $event->getId(), $request->request->get( key: '_token' ))) {
        // Supprime les fichiers images
        foreach ($event->getImages() as $image) {
            $imagePath = $this->getParameter( name: 'events_images_directory' ) . '/' . $image->getUrl();
            if (file_exists($imagePath)) {
                unlink($imagePath);
            }
        }

        $em->remove($event);
        $em->flush();
        $this->addFlash( type: 'success', message: 'Événement supprimé avec succès ! ');
    }

    return $this->redirectToRoute( route: 'app_profile_events' );
}
```

FollowController

Ce contrôleur implémente le système de suivi des utilisateurs et des villes avec les action follow/unfollow

```
#[Route('/user/{id}', name: 'app_follow_user', requirements: ['id' => '\d+', methods: ['POST'])]
public function followUser(
    User $userToFollow,
    Request $request,
    EntityManagerInterface $em,
    UserFollowRepository $userFollowRepository
): Response {
    $currentUser = $this->getUser();

    // On ne peut pas se suivre soi-même
    if ($currentUser == $userToFollow) {
        $this->addFlash( type: 'error', message: 'Vous ne pouvez pas vous suivre vous-même. ');
        return $this->redirectBack($request);
    }

    // Vérifie si déjà suivi
    $existingFollow = $userFollowRepository->findFollow($currentUser, $userToFollow);

    if ($existingFollow) {
        $this->addFlash( type: 'warning', message: 'Vous suivez déjà cet utilisateur. ');
        return $this->redirectBack($request);
    }

    // Crée le follow
    $follow = new UserFollow();
    $follow->setFollower($currentUser);
    $follow->setFollowed($userToFollow);
    $follow->setCreatedAt(new \DateTimeImmutable());

    $em->persist($follow);
    $em->flush();

    $this->addFlash( type: 'success', message: 'Vous suivez maintenant ' . $userToFollow->getFirstName() . ' ' . $userToFollow->getLastName());
    return $this->redirectBack($request);
}
```

HomeController

Ce contrôleur gère l'affichage page d'accueil, il utilise un endpoint API pour charger dynamiquement les événements avec une pagination si nécessaire. Les événements affichés dépendent de la localisation de l'utilisateur pour afficher les événements de son département ou de sa ville.

```
#[Route('/api/events', name: 'app_home_events', methods: ['GET'])]
public function loadEvents(
    Request $request,
    EventRepository $eventRepository
): JsonResponse {
    $departmentId = $request->query->get( key: 'department');
    $offset = $request->query->getInt( key: 'offset', default: 0);
    $limit = $request->query->getInt( key: 'limit', default: 3);

    $events = $eventRepository->findByDepartmentWithPromotedFirst($departmentId, $offset, $limit);
    $total = $eventRepository->countByDepartment($departmentId);

    $now = new \DateTimeImmutable();
    $eventsData = [];
    foreach ($events as $event) {
        // Vérifier si l'événement est promu actuellement
        $promote = $event->getPromote();
        $isPromoted = $promote && $promote->getDateStart() <= $now && $promote->getDateEnd() >= $now;

        $eventsData[] = [
            'id' => $event->getId(),
            'title' => $event->getTitle(),
            'city' => $event->getCity()->getName(),
            'description' => $event->getDescription(),
            'departmentCode' => $event->getCity()->getDepartment()->getCode(),
            'dateStart' => $event->getDateStart()->format('d/m/Y à H:i'),
            'isPromoted' => $isPromoted,
            'image' => $event->getImages()->count() > 0
                ? '/uploads/events/' . $event->getImages()->first()->getUrl()
                : '/images/placeholder.jpg',
            'categories' => array_map(fn($cat) => $cat->getName(), $event->getCategories()->toArray()),
        ];
    }

    return new JsonResponse([
        'events' => $eventsData,
        'hasMore' => ($offset + $limit) < $total,
        'total' => $total,
    ]);
}
```

6.1.5. Interface d'administration (easyAdmin)

L'interface d'administration a été générée à l'aide du bundle EasyAdmin, permettant de gérer rapidement les utilisateurs, départements, villes, catégories, et demandes de “mise en avant”

Le **DashboardController** constitue le point d'entrée de l'administration. il configure le menu latéral et affiche un compteur de demande de “mise en avant” en attente :

```
#[IsGranted('ROLE_ADMIN')]
public function index(): Response
{
    $adminUrlGenerator = $this->container->get(AdminUrlGenerator::class);
    return $this->redirect($adminUrlGenerator->setController(UserCrudController::class)->generateUrl());
}

no usages  Laurent CHIPAUX
public function configureDashboard(): Dashboard
{
    return Dashboard::new()
        ->setTitle( title: 'Administration');
}

no usages  Laurent CHIPAUX
public function configureMenuItems(): iterable
{
    $pendingCount = $this->promoteRequestRepository->countPending();

    yield MenuItem::linkToDashboard( label: 'Dashboard', icon: 'fa fa-home');

    yield MenuItem::section( label: 'Utilisateurs');
    yield MenuItem::linkToCrud( label: 'Utilisateurs', icon: 'fas fa-users', entityFqcn: User::class);

    yield MenuItem::section( label: 'Localisation');
    yield MenuItem::linkToCrud( label: 'Départements', icon: 'fas fa-map', entityFqcn: Department::class);
    yield MenuItem::linkToCrud( label: 'Villes', icon: 'fas fa-city', entityFqcn: City::class);

    yield MenuItem::section( label: 'Événements');
    yield MenuItem::linkToCrud( label: 'Catégories', icon: 'fas fa-tags', entityFqcn: Category::class);
    yield MenuItem::linkToCrud( label: 'Événements', icon: 'fas fa-calendar', entityFqcn: Event::class);

    yield MenuItem::section( label: 'Promotions');
    yield MenuItem::linkToCrud(
        label: 'Demandes' . ($pendingCount > 0 ? ' (' . $pendingCount . ')' : ''),
        icon: 'fas fa-star',
        entityFqcn: PromoteRequest::class
    );
}
```

Le **PromoteRequestController** contient une logique métier importante, lors de l'approbation d'une demande par l'administration, il crée automatiquement l'entité Promote qui active la mise en avant de l'événement.

```
public function updateEntity(EntityManagerInterface $em, $entityInstance): void
{
    /** @var PromoteRequest $entityInstance */

    if ($entityInstance->getStatus() === PromoteRequest::STATUS_APPROVED) {
        $event = $entityInstance->getEvent();

        if (!$event->getPromote()) {
            $promote = new Promote();
            $promote->setEvent($event);
            $promote->setDateStart(\DateTimeImmutable::createFromInterface($entityInstance->getDateStart()));
            $promote->setDateEnd(\DateTimeImmutable::createFromInterface($entityInstance->getDateEnd()));
            $promote->setPrice($entityInstance->getTotalPrice());

            $em->persist($promote);
        }

        $entityInstance->setProcessedAt(new \DateTimeImmutable());
    } elseif ($entityInstance->getStatus() === PromoteRequest::STATUS_REJECTED) {
        $entityInstance->setProcessedAt(new \DateTimeImmutable());
    }

    parent::updateEntity($em, $entityInstance);
}
```

6.1.6. Requête personnalisées (Repository)

Les repositories permettent d'encapsuler les requêtes complexes à la base de données. Chaque entité dispose de son propre repository qui hérite de "ServiceEntityRepository". Nous avons développé plusieurs méthodes personnalisées utilisant le QueryBuilder de Doctrine.

EventRepository

Ce repository est le plus complet car il gère la logique métier principale de l'application. Il contient plusieurs méthodes de requête avancées.

La méthode "findWithFilters" permet la recherche multicritères sur les événements :

```
public function findWithFilters(
    ?int $cityId = null,
    ?int $departmentId = null,
    ?int $categoryId = null,
    ?string $dateFrom = null,
    ?string $dateTo = null,
    ?bool $freeOnly = false,
    ?string $search = null
): array {
    $qb = $this->createQueryBuilder( alias: 'e')
        ->leftJoin( join: 'e.city', alias: 'c')
        ->leftJoin( join: 'c.department', alias: 'd')
        ->leftJoin( join: 'e.categories', alias: 'cat')
        ->leftJoin( join: 'e.promote', alias: 'p')
        ->addSelect( ...select: 'CASE WHEN p.id IS NOT NULL AND p.dateStart <= :now AND p.dateEnd >= :now THEN 1 ELSE 0 END AS HIDDEN isPromoted')
        ->andWhere( ...where: 'e.dateStart >= :now')
        ->setParameter( key: ':now', new \DateTimeImmutable())
        ->orderBy( sort: 'isPromoted', order: 'DESC')
        ->addOrderBy( sort: 'e.dateStart', order: 'ASC');

    if ($cityId) {
        $qb->andWhere( ...where: 'c.id = :cityId')
            ->setParameter( key: ':cityId', $cityId);
    }

    if ($departmentId) {
        $qb->andWhere( ...where: 'd.id = :departmentId')
            ->setParameter( key: ':departmentId', $departmentId);
    }

    if ($categoryId) {
        $qb->andWhere( ...where: 'cat.id = :categoryId')
            ->setParameter( key: ':categoryId', $categoryId);
    }

    if ($dateFrom) {
        $qb->andWhere( ...where: 'e.dateStart >= :dateFrom')
            ->setParameter( key: ':dateFrom', new \DateTimeImmutable($dateFrom));
    }

    if ($dateTo) {
        $qb->andWhere( ...where: 'e.dateStart <= :dateTo')
            ->setParameter( key: ':dateTo', new \DateTimeImmutable( datetime: $dateTo . ' 23:59:59'));
    }

    if ($freeOnly) {
        $qb->andWhere( ...where: 'e.price IS NULL OR e.price = 0');
    }

    if ($search) {
        $qb->andWhere( ...where: 'e.title LIKE :search OR e.description LIKE :search')
            ->setParameter( key: ':search', value: '%' . $search . '%');
    }

    return $qb->getQuery()->getResult();
}
```

La méthode “findByDepartmentWithPromotedFirst” affiche les événements sponsorisés en priorité grâce à un tri conditionnel :

```
1 usage  Laurent CHIPPAUX
public function findByDepartmentWithPromotedFirst(?int $departmentId, int $offset = 0, int $limit = 3): array
{
    $qb = $this->createQueryBuilder( alias: 'e')
        ->leftJoin( join: 'e.city', alias: 'c')
        ->leftJoin( join: 'c.department', alias: 'd')
        ->leftJoin( join: 'e.promote', alias: 'p')
        ->addSelect( ...select: 'CASE WHEN p.id IS NOT NULL AND p.dateStart <= :now AND p.dateEnd >= :now THEN 1 ELSE 0 END AS HIDDEN isPromoted')
        ->setParameter( key: 'now', new \DateTimeImmutable())
        ->orderBy( sort: 'isPromoted', order: 'DESC')
        ->addOrderBy( sort: 'e.dateStart', order: 'DESC')
        ->setFirstResult($offset)
        ->setMaxResults($limit);

    if ($departmentId) {
        $qb->andWhere( ...where: 'd.id = :departmentId')
            ->setParameter( key: 'departmentId', $departmentId);
    }

    return $qb->getQuery()->getResult();
}
```

Les méthodes pour le fil d’actualité personnalisé récupèrent les événements des villes et des créateurs suivis :

```
public function findByFollowedCities(User $user, int $limit = 10): array
{
    return $this->createQueryBuilder( alias: 'e')
        ->innerJoin( join: 'e.city', alias: 'c')
        ->innerJoin( join: 'App\Entity\UserCity', alias: 'uc', conditionType: 'WITH', condition: 'uc.city = c')
        ->andWhere( ...where: 'uc.user = :user')
        ->andWhere( ...where: 'e.dateStart >= :now')
        ->setParameter( key: 'user', $user)
        ->setParameter( key: 'now', new \DateTimeImmutable())
        ->orderBy( sort: 'e.dateStart', order: 'ASC')
        ->setMaxResults($limit)
        ->getQuery()
        ->getResult();
}

1 usage  Laurent CHIPPAUX
public function findByFollowedUsers(User $user, int $limit = 10): array
{
    return $this->createQueryBuilder( alias: 'e')
        ->innerJoin( join: 'e.createdBy', alias: 'u')
        ->innerJoin( join: 'App\Entity\UserFollow', alias: 'uf', conditionType: 'WITH', condition: 'uf.followed = u')
        ->andWhere( ...where: 'uf.follower = :user')
        ->andWhere( ...where: 'e.dateStart >= :now')
        ->setParameter( key: 'user', $user)
        ->setParameter( key: 'now', new \DateTimeImmutable())
        ->orderBy( sort: 'e.dateStart', order: 'ASC')
        ->setMaxResults($limit)
        ->getQuery()
        ->getResult();
}
```

UserFollowRepository

Ce repository gère les relations de suivi entre utilisateurs

```
public function findFollow(User $follower, User $followed): ?UserFollow
{
    return $this->findOneBy([
        'follower' => $follower,
        'followed' => $followed,
    ]);
}
```

1 usage  Laurent CHIPAUX

```
public function findFollowedUsers(User $user): array
{
    return $this->createQueryBuilder( alias: 'uf')
        ->innerJoin( join: 'uf.followed', alias: 'u')
        ->andWhere( ...where: 'uf.follower = :user')
        ->setParameter( key: 'user', $user)
        ->orderBy( sort: 'uf.createdAt', order: 'DESC')
        ->getQuery()
        ->getResult();
}
```

no usages  Laurent CHIPAUX

```
public function findFollowers(User $user): array
{
    return $this->createQueryBuilder( alias: 'uf')
        ->innerJoin( join: 'uf.follower', alias: 'u')
        ->andWhere( ...where: 'uf.followed = :user')
        ->setParameter( key: 'user', $user)
        ->orderBy( sort: 'uf.createdAt', order: 'DESC')
        ->getQuery()
        ->getResult();
}
```

6.1.7. Gestion de l'upload des images

Les images des événements sont uploadées dans le dossier public/uploads/events. Le **ProfileController** gère le renommage sécurisé des fichiers via le “SluggerInterface” de Symfony et la suppression physique des fichiers lors de la suppression d’une image ou d’un événement.

```
// Gestion des images
$imageFiles = $form->get('images')->getData();
if ($imageFiles) {
    $position = 0;
    foreach ($imageFiles as $imageFile) {
        $originalFilename = pathinfo($imageFile->getClientOriginalName(), flags: PATHINFO_FILENAME);
        $safeFilename = $slugger->slug($originalFilename);
        $newFilename = $safeFilename . '-' . uniqid() . '.' . $imageFile->guessExtension();

        try {
            $imageFile->move(
                $this->getParameter( name: 'events_images_directory'),
                $newFilename
            );

            $eventImage = new EventImage();
            $eventImage->setUrl($newFilename);
            $eventImage->setPosition($position);
            $eventImage->setCreatedAt(new \DateTimeImmutable());
            $eventImage->setEvent($event);

            $em->persist($eventImage);
            $position++;
        } catch (FileNotFoundException $e) {
            $this->addFlash( type: 'error', message: 'Erreur lors de l\'upload d\'une image.' );
        }
    }
}
}
```

6.1.8. Test Unitaires.

Des tests unitaires ont été mis en place avec PHPUnit pour valider le comportement des entités et la logique métier. Ces tests permettent de s’assurer que le code fonctionne correctement et de détecter les régressions lors des modifications futures.

Test de l’entité User

Le fichier “UserTest.php” vérifie le comportement de l’entité User, notamment l’attribution automatique du rôle ROLE_USER et de la méthode __ToString()

```
class UserTest extends TestCase
{
    //Test que ROLE_USER est ajouté automatiquement
    ⚡ Laurent CHIPPAUX
    public function testRoleUserAlwaysAdd(): void
    {
        $user = new User();
        $this->assertContains( needle: 'ROLE_USER', $user->getRoles());
    }

    //Test méthode __toString
    ⚡ Laurent CHIPPAUX
    public function testToStringReturnsFullName(): void{
        $user = new User();
        $user->setFirstName( firstName: 'John');
        $user->setLastName( lastName: 'Doe');
        $this->assertEquals( expected: 'John Doe', (string)$user);
    }
}
```

Test de l'entité Event

Le fichier "EventTest.php" vérifie le comportement de l'entité Event :

```
class EventTest extends TestCase
{
    //Test de la méthode toString
    ⚡ Laurent CHIPAUX
    public function testToStringReturnsTitle(): void
    {
        $event = new Event();
        $event->setTitle( title: 'Concert de jazz');

        $this->assertEquals( expected: 'Concert de jazz', (string) $event);
    }
}
```

Test des filtres

Le fichier FilterTest.php vérifie que les paramètres de filtrage sont correctement extraits de la requête HTTP. Ce test valide la logique utilisée dans le contrôleur EventController :

```
class FilterTest extends TestCase{

    //Test que les filtres sont correctement extrait de la requête
    ⚡ Laurent CHIPAUX
    public function testFilterExtractedFromRequest(){
        $request = new Request([
            'city' => '42',
            'department' => '69',
            'category' => '3',
            'date_from' => '2025-01-01',
            'date_to' => '2025-12-31',
            'free_only' => '1',
            'search' => 'concert',
        ]);

        $this->assertEquals( expected: '42', $request->query->get( key: 'city'));
        $this->assertEquals( expected: '69', $request->query->get( key: 'department'));
        $this->assertEquals( expected: '3', $request->query->get( key: 'category'));
        $this->assertEquals( expected: '2025-01-01', $request->query->get( key: 'date_from'));
        $this->assertEquals( expected: '2025-12-31', $request->query->get( key: 'date_to'));
        $this->assertTrue($request->query->getBoolean( key: 'free_only'));
        $this->assertEquals( expected: 'concert', $request->query->get( key: 'search'));
    }
}
```

Pour aller plus loin on pourrait ajouter de nombreux test dont des test pour les contrôleurs en simulant des requêtes HTTP, des test de validations sur les formulaires, des test liés à la base de données. Étant donné que le site n'est pas conçu pour une mise en production puisque c'est un projet de cours, nous n'avons pas créé plus de tests.

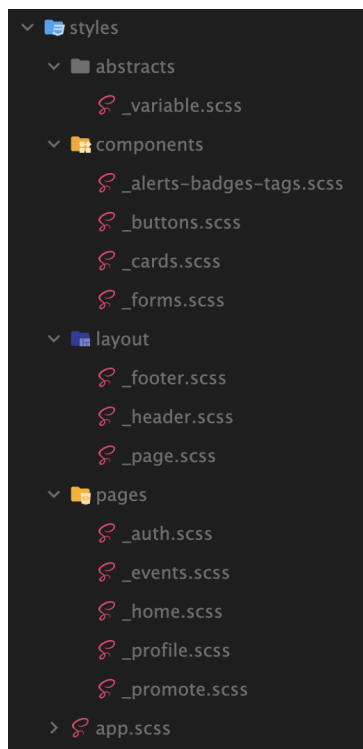
6.2. Développement Front-end

6.2.1. Technologies utilisées

Le front-end de l'application repose sur les technologies suivantes :

- **Moteur de template Twig** : permettant une séparation claire entre la logique métier et la présentation. Twig offre un système d'héritage de templates et de blocs facilitant la réutilisation du code.
- **SCSS** : Préprocesseur CSS permettant d'utiliser des variables, l'imbrication de sélecteurs et l'importation modulaire. Le SCSS est compilé en CSS via le File Watcher de PHPStorm.
- **JavaScript** : Pour les interactions dynamiques et l'utilisation de l'api IP.API pour la localisation de l'utilisateur

6.2.2. Architecture SCSS



L'organisation des fichiers SCSS suit une architecture modulaire.

Fichier _variables.scss :

```
//Colors
$color-orange : #D2691E;
$color-orange-hover : #E07856;
$color-yellow : #FFD700;

$color-gris : #2D3748;
$color-black : #1A1F29;
$color-gris-clair : #E6E8EB;

$color-success: #38d56c;
$color-warning: #f5d000;
$color-error: #dc2626;
$color-info: #4facfe;

// Font families
$font-primary: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
$font-logo: 'Vast Shadow', cursive;

// Breakpoint
$breakpoint-sm: 640px;
$breakpoint-md: 768px;
$breakpoint-lg: 1024px;
$breakpoint-xl: 1280px;
$breakpoint-2xl: 1536px;

//Container
$container-max-width: 1280px;
$container-padding: 1rem;
$container-narrow: 720px;
```

6.2.3. Template Twig de base (base.html.twig)

Ce template utilise le système de blocs Twig permettant aux pages enfants de surcharger le titre, d'ajouter des styles ou scripts spécifiques, et de définir leur contenu principal.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}La Vie en Rose{% endblock %}</title>

  {# Google Fonts #}
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=Vast+Shadow&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css">

  <link rel="stylesheet" href="{{ asset('styles/app.css') }}">
</head>
```

```
<header class="header">
  <div class="header__container">
    <a href="{{ path('app_home') }}" class="header__logo logo">
      <span class="logo__text">La Vie<br>en Rose</span>
    </a>

    <nav class="header__nav nav">
      <a href="{{ path('app_home') }}" class="nav__link">Accueil</a>
      <a href="{{ path('app_event_index') }}" class="nav__link">Événements</a>

      {% if app.user %}
        <div class="nav__user user-menu">
          <button class="user-menu__trigger" type="button" aria-expanded="false">
            <span class="user-menu__avatar avatar">
              {{ app.user.firstName|first|upper }}{{ app.user.lastName|first|upper }}
            </span>
          </button>

          <div class="user-menu__dropdown">
            {% if is_granted('ROLE_ADMIN') %}
              <a href="{{ path('admin') }}" class="user-menu__item">Administration</a>
            {% endif %}
            <a href="{{ path('app_profile') }}" class="user-menu__item">Mon profil</a>
            <a href="{{ path('app_profile_events') }}" class="user-menu__item">Mes événements</a>
            <a href="{{ path('app_my_follows') }}" class="user-menu__item">Mes abonnements</a>
            <hr class="user-menu__divider">
            <a href="{{ path('app_logout') }}" class="user-menu__item user-menu__item--logout">Déconnexion</a>
          </div>
        </div>
      {% else %}
        <a href="{{ path('app_login') }}" class="nav__link nav__link--cta btn btn--primary btn--sm">Connexion</a>
      {% endif %}
    </nav>

    {# Mobile menu toggle #}
    <button class="header__mobile-toggle" type="button" aria-label="Menu">
      <span class="hamburger"></span>
    </button>
  </div>
</header>
```

```

<main class="main">
    {# Flash messages #}
    {% for label, messages in app.flashes %}
        {% for message in messages %}
            <div class="alert alert--{{ label }}">
                <span class="alert_message">{{ message }}</span>
                <button class="alert_close" type="button" aria-label="Fermer">&times;</button>
            </div>
        {% endfor %}
    {% endfor %}

    {% block body %}{% endblock %}
</main>

```

```

<footer class="footer">
    <div class="footer_container">
        <div class="footer_links">
            <a href="#" class="footer_link">À propos</a>
            <a href="#" class="footer_link">Contact</a>
            <a href="#" class="footer_link">CGU / Confidentialité</a>
        </div>
        <p class="footer_copyright">&copy; {{ "now"|date("Y") }} La Vie en Rose – Réseau Social d'Événements</p>
    </div>
</footer>
{% block javascripts %}{% endblock %}

```

6.2.4. Composant twig réutilisable

_card_event.html.twig : Composant de card événement réutilisable sur la page liste, l'accueil et le tableau de bord.

```

{% set is_promoted = event.promote and event.promote.dateStart <= date() and event.promote.dateEnd >= date() %}

<article class="event-card {{ is_promoted ? 'event-card--promoted' : '' }} {{ compact|default(false) ? 'event-card--compact' : '' }}">
    {# Badge sponsorisé #}
    {% if is_promoted %}
        <span class="event-card_badge badge badge--promoted"><i class="fa-solid fa-star"></i> Sponsorisé</span>
    {% endif %}

    {# Image #}
    <div class="event-card_image">
        {% if event.images is not empty and event.images|first.url %}
            
        {% else %}
            {% set gradients = [
                'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',
                'linear-gradient(135deg, #f093fb 0%, #f5576c 100%)',
                'linear-gradient(135deg, #4facfe 0%, #00f2fe 100%)',
                'linear-gradient(135deg, #43e97b 0%, #38f9d7 100%)',
                'linear-gradient(135deg, #fa709a 0%, #fee140 100%)',
                'linear-gradient(135deg, #30cfd0 0%, #330867 100%)',
                'linear-gradient(135deg, #a8edea 0%, #fed6e3 100%)',
                'linear-gradient(135deg, #ff9a9e 0%, #fecfef 100%)'
            ] %}
            <div class="event-card_placeholder" style="background: {{ gradients[event.id % 8] }};">
                <span class="event-card_placeholder-icon"><i class="fa-regular fa-calendar-days"></i></span>
            </div>
        {% endif %}
    </div>

```

```

{# Content #}
<div class="event-card_content">
  <h3 class="event-card_title">{{ event.title }}</h3>

  <div class="event-card_meta">
    <span class="event-card_location">
      <span class="event-card_icon"><i class="fa-solid fa-map-location-dot"></i></span>
      {{ event.city.name }} {{ event.city.department.code }}
    </span>
    <span class="event-card_date">
      <span class="event-card_icon"><i class="fa-regular fa-calendar-days"></i></span>
      {{ event.dateStart|date('d/m/Y à H\\hi') }}
    </span>
    <span class="event-card_price">{{ event.price is null or event.price == 0 ? 'event-card_price--free' : '' }}</span>
    {% if event.price is not null and event.price > 0 %}
      <span class="event-card_icon"><i class="fa-solid fa-hand-holding-dollar"></i></span> {{ event.price|number_format(2, ',', ' ') }}€
    {% else %}
      <span class="event-card_icon"><i class="fa-regular fa-star"></i></span> Gratuit
    {% endif %}
  </div>
</div>

{# Categories #}
{% if event.categories is not empty %}
  <div class="event-card_tags">
    {% for category in event.categories|slice(0, 3) %}
      <span class="tag">{{ category.name }}</span>
    {% endfor %}
  </div>
{% endif %}

{# Description (truncated) #}
{% if event.description and not (compact|default(false)) %}
  <p class="event-card_description">
    {{ event.description|length > 120 ? event.description|slice(0, 120) ~ '...' : event.description }}
  </p>
{% endif %}

{# Actions #}
<div class="event-card_actions">
  <a href="{{ path('app_event_show', {id: event.id}) }}" class="btn btn--primary btn--sm">
    Voir les détails
  </a>

  {% if show_actions|default(false) and app.user == event.createdBy %}
    <a href="{{ path('app_profile_event_edit', {id: event.id}) }}" class="btn btn--outline btn--sm" title="Modifier">
      
    </a>
  {% endif %}
</div>
</div>
</article>

```

7. Déploiement

7.1. Conteneurisation avec Docker

Docker nous a permis d'uniformiser les environnements de développement entre les trois membres de l'équipe et de garantir un déploiement identique en production. Notre infrastructure repose sur trois conteneurs orchestrés via Docker Compose.

Dockerfile

```
FROM php:8.2-fpm

RUN apt-get update && apt-get install -y \
    git unzip libicu-dev libzip-dev libsodium-dev \
    && docker-php-ext-install pdo pdo_mysql intl zip sodium

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html
```

Les extensions installées répondent aux besoins de Symfony

- **pdo_mysql** : connexion à la base de données MySQL
- **intl** : Internationalisation (dates, nombres, etc)
- **zip** : manipulation des archives (utilisé par Composer)
- **sodium** : fonctions cryptographiques pour la sécurité

Docker Compose

```
services:
  php:
    build: .
    container_name: symfony-php
    environment:
      DEFAULT_URI: http://164.132.226.60:8081/
    working_dir: /var/www/html
    volumes:
      - ./:/var/www/html
    networks:
      - symfony

  nginx:
    image: nginx:latest
    container_name: symfony_nginx
    ports:
      - "8081:80"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d
      - ./:/var/www/html
    depends_on:
      - php
    networks:
      - symfony
```

```
db:
  image: mysql:8.0
  container_name: symfony_db
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  ports:
    - "3307:3306"
  volumes:
    - db_data:/var/lib/mysql
    - ./docker/mysql-init:/docker-entrypoint-initdb.d
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s
  networks:
    - symfony

volumes:
  db_data:

networks:
  symfony:
```

7.2. Pipeline CI/CD avec GitLab

Nous avons mis en place une pipeline d'intégration et de déploiement continu (CI/CD) sur GitLab. Cette pipeline automatise le processus de build, de test et de déploiement à chaque push sur les branches principales.

Configuration de la pipeline

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  image: php:8.2-cli
  before_script:
    - apt-get update && apt-get install -y unzip git
    - curl -sS https://getcomposer.org/installer | php
  script:
    - php composer.phar install --no-interaction
  artifacts:
    paths:
      - vendor/

test:
  stage: test
  image: php:8.2-cli
  dependencies:
    - build
  script:
    - php tests/unit/tests/unit/EventTest.php
    - php tests/unit/FilterTest.php
    - php tests/unit/UserTest.php

deploy_to_vps:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache sshpass openssh

  script:
    - sshpass -p $VPS_PASSWORD ssh $VPS_USER@$VPS_HOST "
      cd /srv/reseau_social_ville &&
      git pull &&
      docker compose pull &&
      docker compose up -d --remove-orphans
    "

  only:
    - main
    - test-déploiement
```

Étapes de la pipeline :

- installation des dépendances
- exécution des tests
- Déploiement en production

Installation des outils nécessaires
Téléchargement de Composer

Installation des dépendances

Récupère le dossier vendor/ de l'étape
build

Exécution des tests unitaires PHPUnit

Installation des outils SSH

Connexion SSH au serveur et
exécution des commandes de
déploiement

Déclenché uniquement sur les
branches main et de test

7.3. Processus de déploiement

Lorsqu'un développeur pousse du code sur la branche main, le processus suivant s'exécute automatiquement :

- **Build:** GitLab télécharge le code et installe les dépendances Composer
- **Test:** Les tests unitaires sont exécutés pour vérifier que le code fonctionne
- **Déploy :** Si les tests passent, GitLab se connecte au VPS via SSH et exécute :
 - *git pull* : récupère les dernières modifications
 - *docker compose pull* : télécharge les nouvelles images si nécessaire
 - *docker compose up -d --remove-orphans* : redémarre les conteneurs

7.4. Bilan du déploiement

Cette infrastructure nous a permis de :

- **Uniformiser les environnements :** tous les développeurs travaillent avec la même configuration
- Automatiser les tests : Chaque modification est testée avant le déploiement
- Simplifier le déploiement : un simple push déclenche la mise en production
- Réduire les erreurs humaines : pas d'intervention manuelle nécessaire

8. Conclusion

Ce projet de réseau social d'événements locaux m'a permis de mettre en pratique l'ensemble des compétences attendues pour le titre professionnel Concepteur Développeur d'Applications, en couvrant toutes les phases du cycle de développement logiciel.

D'un point de vue technique, j'ai pu approfondir mes connaissances sur le framework Symfony et son écosystème : Doctrine ORM pour la gestion des données, le composant Security pour l'authentification et les autorisations, EasyAdmin pour la création rapide d'un back-office fonctionnel, ainsi que PHPUnit pour les tests unitaires. La mise en place d'une architecture Docker couplée à une pipeline CI/CD GitLab m'a également permis de découvrir les bonnes pratiques du déploiement continu.

Le travail en équipe de trois développeurs a été une expérience enrichissante. Cette organisation nous a confronté aux réalités du travail en équipe : communication, répartition des tâches, résolution des conflits de code et respect des délais. Nous avons eu pas mal de problèmes de communication.

Bien que fonctionnelle, l'application pourrait être enrichie par plusieurs évolutions : l'intégration d'un système de paiement en ligne, amélioration des tests ou encore mise en place d'une carte interactive pour afficher les événements.

Ce projet représente une synthèse concrète de ma formation en tant que Concepteur Développeur d'Applications. Il m'a permis de consolider mes acquis techniques tout en développant ma capacité à travailler en équipe. Ces compétences seront directement utilisables dans mon activité professionnelle au sein de l'agence "La Couleur du Zèbre" et dans mes futurs projets de développement.